Calibo AI Academy · Path 2 · KLU Batch

Phase 2 — Mini Use Case 02

Credit Pre-Screening Engine — Loan Intake Optimisation

Module: Supervised Learning Part 1 (Regression & Problem Framing) | Industry: BFSI — Retail Lending

1. A Note Before You Begin

There is a quote I carry with me, and I want to share it with you again before you start:

> ***"The day you understand the value of the work you do, is the day you add more value to the work you do."***

In your first mini use case, you looked back at what had already happened. This time you build something that makes a decision going forward. A model that predicts loan outcomes is not an academic exercise — every threshold you choose decides whether a real person's application is read by a human or turned away by a machine.

Your job is not to build the most accurate model. Your job is to build the most useful one — a model the Credit Policy Analyst can defend to her Head of Retail Lending, to the analysts whose work it changes, and to the customers it affects. Keep all three in mind as you work.

2. Use Case Overview

Field	Details
Title	Credit Pre-Screening Engine — Loan Intake Optimisation
Industry	BFSI — Retail Lending (Personal Loans)
Phase & Module	Phase 2 — Supervised Learning Part 1
Batch	Path 2 — KLU
Structure	Single use case, seven steps
Assignment type	Formative — Mini Use Case
Tools	Python · Jupyter Notebook · Pandas · NumPy · Matplotlib · Seaborn · scikit-learn
Dataset	MUC02_Credit_PreScreening_Dataset.csv (provided alongside this document)
Submission	Jupyter Notebook (.ipynb) + 1-page written insight summary
Deadline	To be confirmed by your trainer

3. Business Context

A mid-size private bank runs a retail lending unit that offers unsecured personal loans to customers across India — salaried employees, self-employed professionals, small business owners and gig workers, in metros as well as Tier 2 and Tier 3 cities.

The unit receives roughly 3,000 personal loan applications a month. Every one of them goes through a manual credit review that takes an analyst about 25 minutes. Close to half of all applications end up declined — and a large share of

those are decided within the first minute, because the applicant's credit score is very low or their existing EMIs already consume most of their income.

The Head of Retail Lending has asked the Credit Policy Analyst a direct question: can we use our own history to build a pre-screening model, so analysts spend their time on applications that genuinely need judgement? She has six months of past applications with the final decision recorded for each one. She needs you to tell her what the data says and how far a model can safely be trusted.

> **The business question you are answering:** Using six months of historical decisions, can a pre-screening model safely remove clear-cut declines from the manual review queue — and how many applications, and how many analyst hours, would that free up each month without wrongly turning away creditworthy customers?

Specifically, the Credit Policy Analyst needs to know:

What she needs	Why it matters
What actually drives approval decisions today	To check that the model learns the bank's real credit policy — not noise
How well a model can predict the decision	To judge whether a model is ready to touch live applications at all
A safe auto-decline threshold	To control the risk of turning away good customers
How much analyst time is freed up	To put a business number on the proposal for the Head of Retail Lending
Whether the model treats applicants fairly	To make sure the bank is not automating a bias it cannot defend

4. Dataset

You have been provided with a simulated dataset: MUC02_Credit_PreScreening_Dataset.csv. Place it in the same folder as your Jupyter Notebook before you start.

> **Local setup:** All work runs on your own laptop. Make sure you have the following installed:
> pip install pandas numpy matplotlib seaborn scikit-learn jupyter
> To start, open a terminal, navigate to your working folder, and run: jupyter notebook

The dataset has 12,000 loan applications received between April and September 2026. Each row is one application, with the final decision made by the bank's credit analysts. Column reference:

Column	Description
application_id	Unique identifier for each application
application_date	Date the application was received (YYYY-MM-DD, Apr–Sep 2026)
age	Applicant age in years
gender	Applicant gender as recorded on the application form
city / city_tier	City of residence, and its tier (1 = metro, 2 = large city, 3 = smaller city)
employment_type	Salaried, Self-Employed Professional, Business Owner, or Gig / Contract
years_in_current_job	Years in current job or business

Column	Description
monthly_income	Declared gross monthly income in INR
existing_emi	Total monthly EMIs the applicant already pays on existing loans, in INR
credit_score	Credit bureau score, 300–900. Blank where the applicant has no credit history
credit_history_months	Length of the applicant's credit history in months (0 = no history)
num_active_loans	Number of loans currently active
delinquencies_24m	Number of late or missed repayments in the last 24 months
loan_purpose	Stated purpose of the loan
loan_amount	Loan amount requested, in INR
tenure_months	Requested repayment period in months
proposed_emi	Monthly EMI the new loan would add, in INR (calculated at 14% p.a.)
decision	Final decision by the credit team — Approved or Declined. This is your target.

Load the dataset with: df = pd.read_csv("MUC02_Credit_PreScreening_Dataset.csv")

> **A term you will need — FOIR.** Indian lenders measure affordability using the Fixed Obligation to Income Ratio: the share of monthly income that goes to EMIs. FOIR = (existing EMIs + proposed EMI) ÷ monthly income. A FOIR of 0.60 means 60% of the applicant's income would go to loan repayments. You will calculate it yourself in Step 5.3.

5. What You Will Build

Your Jupyter Notebook should work through the steps below in sequence. Each step has two parts — the code, and a markdown cell in plain English explaining what you did and what you found. The markdown cells are not optional; they are half the assignment.

5.1 Load, inspect and clean

Check the shape, data types and null values. Calculate the overall approval rate — this tells you how balanced your target is. Then look closely at three things and decide how to handle each one:

- monthly_income has missing values. Which employment types are they concentrated in? Choose a sensible way to fill them and justify it.
- credit_score is blank for some applicants. Check credit_history_months for those rows. Are these errors, or do they mean something? Do not drop them.
- Look at the maximum of monthly_income. A small number of values look implausible for a monthly figure. What is the most likely explanation, and what will you do about them?

In your markdown, record every cleaning decision and the reason for it. A reviewer at a bank should be able to read your notes and agree with each choice.

5.2 What drives approval today?

Before building any model, find out what the credit team actually does. Group credit_score into bands — below 600, 600–649, 650–699, 700–749, 750–799 and 800+ — plus a separate "No score" group, and plot the approval rate for each. Then plot the approval rate by employment_type.

In your markdown, answer: is there a score below which applications are almost never approved? What does that tell you about the bank's unwritten credit policy? How do applicants with no credit score fare compared to scored applicants?

5.3 Engineer the affordability features

Create three new columns:

- foir = (existing_emi + proposed_emi) / monthly_income
- loan_to_income = loan_amount / (monthly_income × 12)
- is_ntc — 1 if the applicant has no credit score (new to credit), otherwise 0

Group FOIR into bands — below 30%, 30–40%, 40–50%, 50–55% and above 55% — and plot the approval rate for each band.

In your markdown, answer: at what FOIR level do approvals fall sharply? If you were writing this into a credit policy, what FOIR limit would you propose?

5.4 Build the logistic regression pre-screener

Set your target as 1 if the application was Declined, 0 if Approved — the model is predicting declines. Use these features: credit_score, is_ntc, foir, loan_to_income, delinquencies_24m, num_active_loans, years_in_current_job, age, tenure_months, city_tier, employment_type and loan_purpose.

- Do not use application_id, application_date, city, gender or decision as features.
- Fill blank credit scores with the median — the is_ntc flag tells the model which ones were filled.
- One-hot encode employment_type and loan_purpose. Scale the numeric features.
- Split 80/20 with stratify=y and random_state=42 so results are comparable across the batch.

Report accuracy, the confusion matrix, and precision and recall for the Declined class. Then list the five features with the largest coefficients.

In your markdown, explain the confusion matrix in business language: how many creditworthy applicants would this model have wrongly declined? Then explain the top coefficients the way you would to a credit analyst — which factors push an application towards decline?

5.5 Set the auto-decline threshold

A model that declines everyone it predicts as "Declined" at the default 0.5 cut-off will turn away many good customers. Instead, the bank will only auto-decline when the model is very confident. Using predict_proba on your test set, build a table for thresholds of 0.5, 0.6, 0.7, 0.8, 0.85, 0.9 and 0.95 showing:

- The percentage of applications that would be auto-declined
- Of those auto-declined, the percentage that were actually approved by the credit team (wrongly declined)

Plot both lines against the threshold on one chart.

The bank's risk appetite is that no more than 5% of auto-declined applications may be ones the credit team would have approved. Choose the threshold that meets this rule while auto-declining as many applications as possible.

In your markdown, translate your threshold into the monthly business case: with 3,000 applications a month and 25 minutes per manual review, how many applications and how many analyst hours does it free up?

5.6 Fairness check

You did not use gender as a feature. That does not automatically make the model fair. Check three things:

- The historical approval rate by gender, overall and within each employment type
- The median monthly income and the share of Gig / Contract workers by gender
- The auto-decline rate by gender from your model at the threshold you chose in Step 5.5

In your markdown, answer in 3–4 sentences: is there a gap in the historical data? What explains it? Does your model widen it, narrow it, or leave it unchanged? Should gender ever be used as a feature in a credit model — and does leaving it out fully solve the problem?

5.7 Your briefing to the Credit Policy Analyst

Close your notebook with a markdown cell titled "Credit Policy Briefing." Write 4–6 sentences you would actually say to the Credit Policy Analyst before she meets the Head of Retail Lending. Cover:

- The threshold you recommend, and what it means in applications and analyst hours per month
- The two factors that drive decisions most strongly, with a number from your analysis for each
- One group of applicants where the model should not be trusted yet, and why
- Your position on fairness, and one additional piece of data you would want before going live

> This section is evaluated differently from the rest. Technical correctness gets you to a pass. The quality of your business thinking here is what gets you to distinction.

6. CBIM Problem Canvas

Every mini use case begins with a Problem Canvas — completed before you write a single line of code.

> Complete this canvas before touching the data. It should take 15–20 minutes. If you cannot answer these questions clearly, you do not yet understand the problem well enough to model it.

Canvas element	Your answer
Situation (S)	Describe the bank's lending unit and how applications are reviewed today, in 1–2 sentences.
Complication (C)	What problem does the current process create — for the bank, for the analysts, and for applicants?
Question (Q)	What specific decision does this model enable? Not "build a model" — what will the bank do differently?
SSOT	What is the single source of truth for this analysis, and what does the decision column actually represent?
Success definition	How will you know the pre-screener is good enough? Name the metric and the limit that matter.
Primary stakeholder	Who is this for, and who else is affected by the decision she makes?

Submit the completed canvas as a clearly formatted section at the top of your notebook — labelled "CBIM Problem Canvas" — or as a separate document alongside your notebook.

7. Deliverables

If you are working individually

What to submit	Details
Jupyter Notebook	One .ipynb file covering all seven steps. Every step has both runnable code and a markdown explanation. Runs end-to-end without errors.
CBIM Problem Canvas	Completed canvas — at the top of the notebook or as a separate Word document.
Insight summary	1-page Word document (max 400 words): three key findings, your recommendation, and one limitation.

How your work will be evaluated

Criterion	Weight	What good looks like
Code quality & correctness	20%	All seven steps complete, runs without errors, clean code, no unused cells.
Data preparation & features	15%	Every cleaning decision is justified. FOIR, loan-to-income and the new-to-credit flag are built correctly.
Model & evaluation	15%	Logistic regression built as specified. Confusion matrix and coefficients explained in business language.
Threshold decision	15%	Threshold chosen against the 5% rule, with a clear table and chart. Impact stated in applications and hours.
Business interpretation	15%	Every chart and result followed by what it means for the credit team — not just what it shows.
Fairness check	5%	Gap measured, explained, and a clear position taken on gender and proxy variables.
Credit Policy Briefing	10%	Specific, data-backed, all four points answered with real figures.
CBIM Problem Canvas	5%	All six fields answered thoughtfully before the analysis was written.

If you are working in a group of 2–3

Role	Sections
Data Analyst	Steps 5.1, 5.2 and 5.3 — cleaning, approval drivers and affordability features. CBIM canvas.
Model Builder	Steps 5.4 and 5.5 — logistic regression, evaluation and threshold.
Policy Synthesiser	Steps 5.6 and 5.7 — fairness check and the Credit Policy Briefing.

For a 2-person team: Person A takes Steps 5.1–5.3 and the CBIM canvas. Person B takes Steps 5.4–5.7. Both review before submitting.

What to submit	Details
Jupyter Notebook	One shared notebook. Each step's markdown credits the team member who built it.

What to submit	Details
CBIM Problem Canvas	One canvas completed together. All members must have contributed.
Insight summary	1-page document (max 500 words for a group) — findings, recommendation, limitation, and who owned which section.
Individual reflection	150-word note from each team member — what you contributed and what you learned from your teammate's section.

8. A Few Things Worth Flagging

What often goes wrong	What to do instead
Rows with a blank credit score are dropped	They are new-to-credit applicants — about 9% of all applications. Keep them and flag them with is_ntc
Accuracy is reported as the headline result	Say how many creditworthy applicants would be wrongly declined — that is the number the bank cares about
The threshold is left at 0.5	Use the threshold table and the 5% rule — the bank sets the risk limit, not the default
Coefficients are listed without interpretation	Say what each one means for an applicant — "every extra missed payment pushes the application towards decline"
Fairness is answered with "we did not use gender, so it is fair"	Check the outcomes by gender, and think about which features could act as a stand-in
CBIM canvas filled in after the analysis	The canvas goes first — it changes how you approach the problem
Notebook not tested end-to-end before submitting	Restart the kernel and run all cells — if it breaks, it is not ready

9. Before You Submit

Individual

- Dataset CSV and notebook in the same local folder
- Required libraries installed: pandas, numpy, matplotlib, seaborn, scikit-learn, jupyter
- CBIM Problem Canvas completed — all six fields — before any code was written
- Notebook runs end-to-end without errors (kernel restart + run all cells done)
- All seven steps present with both code and a markdown explanation in each
- Every cleaning decision in Step 5.1 is written down with a reason
- Step 5.5 states the chosen threshold and the monthly impact in applications and hours
- Step 5.7 Credit Policy Briefing answers all four points with specific figures
- Insight summary submitted (max 400 words)
- Files named: [YourName]_MUC02_KLU_Notebook.ipynb and [YourName]_MUC02_KLU_Summary.docx

Group

- Dataset CSV and notebook in the same folder on the machine used for final assembly
- CBIM canvas completed together — all members contributed, all fields answered

- Each step's markdown credits the team member who built that section
- Notebook runs end-to-end — tested by a member who did not write that section
- Chart style is consistent across all sections — same palette, same labelling convention
- The final briefing references findings from other members' sections specifically
- Individual 150-word reflection submitted separately by each team member
- Files named: [TeamName]_MUC02_KLU_Notebook.ipynb and [TeamName]_MUC02_KLU_Summary.docx